

Paralelização do ACO com CUDA e OpenMP para Seleção de Instâncias em Larga Escala

Raphael A. L. Soares¹, Wagner Victor A. Menezes¹,
Victor Gabriel R. Jácome¹, Daniel Rios Borges¹

¹Instituto de Informática – Universidade Federal de Goiás (UFG)
Caixa Postal 131 – CEP 74001-970 – Goiânia - GO – Brasil

{raphael.alves, wagnervictor, victor.ribeiro, daniel.rios}@discente.ufg.br

Abstract. *Instance selection reduces dataset size while preserving classifier quality, but Ant Colony Optimization (ACO), although effective, has an $O(N^2)$ cost that makes it infeasible at large scale: the sequential C++ baseline of Magalhães et al. (2024) does not scale to large bases. We parallelize ACO-based instance selection in OpenMP (CPU) and CUDA (GPU), running the 1-NN evaluation entirely on the GPU, and assess the impact of thread count, scheduler and block size. On nine datasets ($\leq 5k$ instances) a 24-core OpenMP run beats the GPU, but on a 253,680-instance base CUDA reaches $\sim 69\times$ over the sequential version (vs. $\sim 14\times$ for OpenMP), keeping the same quality ($\sim 23\%$ reduction, $F_1 \approx 0.77$). The GPU advantage is therefore scale-dependent: it dominates once the problem is large enough to amortize its overhead.*

Resumo. *A seleção de instâncias reduz o tamanho de bases de dados preservando a qualidade de classificadores, mas a Otimização por Colônia de Formigas (ACO), embora eficaz, tem custo $O(N^2)$ que a inviabiliza em larga escala: o baseline sequencial em C++ de Magalhães et al. (2024) não escala para grandes bases. Este trabalho paraleliza o ACO de seleção de instâncias em OpenMP (CPU) e CUDA (GPU), com a avaliação 1-NN inteiramente na GPU, e mede o impacto de threads, escalonadores e tamanho de bloco. Em nove bases ($\leq 5k$ instâncias) o OpenMP em 24 núcleos supera a GPU, mas em uma base de 253.680 instâncias o CUDA atinge $\sim 69\times$ sobre o sequencial (contra $\sim 14\times$ do OpenMP), preservando a qualidade ($\sim 23\%$ de redução, $F_1 \approx 0,77$). Conclui-se que o ganho da GPU é dependente de escala: ela domina quando o problema é grande o suficiente para amortizar seu custo fixo.*

1. Introdução

Modelos de aprendizado de máquina dependem de conjuntos de treinamento representativos, e bases reais frequentemente contêm instâncias redundantes ou ruidosas que aumentam o custo computacional sem agregar informação. A *seleção de instâncias* ataca esse problema: busca um subconjunto compacto dos dados que preserve (ou melhore) a acurácia de um classificador, reduzindo tempo e recursos de treinamento [Olvera-López et al. 2010].

A Otimização por Colônia de Formigas (ACO) [Dorigo et al. 2006] é uma metaheurística bioinspirada eficaz para seleção de instâncias quando usada

como método *wrapper*, guiando a escolha de instâncias por um classificador 1-NN [Cover and Hart 1967]. Sua principal limitação é o custo computacional: tanto o cálculo de distâncias quanto a avaliação 1-NN são $O(N^2)$ no número de instâncias N , o que torna o algoritmo caro à medida que a base cresce.

O baseline deste trabalho, Magalhães et al. [Magalhães et al. 2024], reescreveu o ACO de Python para C++ e obteve ganhos expressivos de desempenho mantendo a qualidade. Entretanto, essa versão permanece **sequencial**, e os próprios autores apontam que o alto custo computacional “limita sua escalabilidade, tornando-o inviável para grandes bases de dados”. Existe, portanto, uma lacuna clara: falta uma versão **paralela** do ACO de seleção de instâncias capaz de explorar CPUs multicore e GPUs para viabilizar o algoritmo em larga escala.

Este trabalho preenche essa lacuna. Implementamos o ACO de seleção de instâncias em OpenMP (CPU) [Dagum and Menon 1998] e CUDA (GPU) [Nickolls et al. 2008], movendo a avaliação 1-NN inteiramente para a GPU, e avaliamos o impacto do número de threads, do escalonador de laço e do tamanho de bloco. Nas nove bases pequenas ($\leq 5k$ instâncias) o OpenMP em 24 núcleos supera a GPU, cujo custo fixo não se amortiza; já na base CDC Diabetes de 253.680 instâncias o CUDA atinge $\sim 69\times$ de aceleração sobre o sequencial (contra $\sim 14\times$ do OpenMP), preservando a qualidade da solução ($\sim 23\%$ de redução com $F_1 \approx 0,77$). O resultado central é que *o ganho da GPU é dependente de escala*.

2. Formulação do Problema

Seja um conjunto $X = \{x_1, \dots, x_N\}$ com N instâncias e F atributos. A seleção de instâncias procura um subconjunto $S \subseteq X$ que minimize $|S|$ mantendo a acurácia de um classificador treinado em S . No ACO, cada uma de K formigas constrói uma solução binária (seleciona ou não cada instância) guiada por dois fatores: o *feromônio* τ_i , que acumula a utilidade histórica de incluir a instância i , e a *visibilidade* η_i , derivada da distância média da instância às demais. A probabilidade de seleção é proporcional a $\tau_i^\alpha \eta_i^\beta$. A qualidade de cada solução é medida por um classificador 1-NN aplicado sobre o subconjunto selecionado, e o feromônio é reforçado nas soluções de maior F_1 e evaporado a cada iteração.

O custo computacional concentra-se em dois pontos $O(N^2F)$: o pré-cálculo de distâncias e, sobretudo, a avaliação 1-NN, na qual cada instância de teste busca seu vizinho mais próximo no subconjunto selecionado. Para $N = 253,680$, isso significa da ordem de 10^{10} comparações por iteração — o gargalo que motiva a paralelização.

3. Objetivos

1. Implementar o ACO de seleção de instâncias em três versões — sequencial (baseline), OpenMP (CPU) e CUDA (GPU) — com qualidade equivalente.
2. Superar o baseline sequencial em desempenho, preservando a qualidade da seleção (medida por F_1 e taxa de redução).
3. Avaliar a escalabilidade: em OpenMP, variando o número de threads e o escalonador de laço; em CUDA, variando o tamanho de bloco.
4. Reportar o *speedup* (absoluto e relativo) em bases de tamanhos distintos, incluindo uma base de larga escala (253.680 instâncias).

4. Metodologia

4.1. Algoritmo e implementações

As três versões compartilham o mesmo ACO de seleção guiada por feromônio ($\alpha = \beta = 1$, taxa de evaporação 0,1, $K = 64$ formigas). A construção de soluções é *embarçosamente paralela*: cada formiga escreve apenas em sua fatia da colônia e lê o vetor de probabilidades como dado somente-leitura, com gerador aleatório semeado por (*iteração*, k) para garantir determinismo independente do paralelismo.

Na versão **OpenMP**, paralelizam-se a construção das formigas, o depósito de feromônio (com `atomic` para evitar condições de corrida) e, principalmente, a avaliação 1-NN — o laço quente, que responde por mais de 98% do tempo. Esse laço usa `schedule(runtime)`, permitindo controlar o escalonador via a variável `OMP_SCHEDULE`. Na versão **CUDA**, a avaliação 1-NN roda inteiramente na GPU (`knn_1nn_kernel`: uma thread por instância de teste), eliminando o transporte da colônia $K \times N$ a cada iteração; para N grande, o feromônio é mantido como vetor 1D e as distâncias são calculadas *on-the-fly*, evitando a matriz $N \times N$.

4.2. Configuração experimental

Todos os experimentos rodaram na **mesma máquina**: Intel Core i9-14900K (24 núcleos físicos — 8 *performance* + 16 *efficiency* — e 32 threads) e GPU NVIDIA GeForce RTX 4090 (24 GB). Avaliamos nove bases do baseline (de 299 a 4.981 instâncias) e a base **CDC Diabetes Health Indicators** [Centers for Disease Control and Prevention 2015] (253.680 instâncias, 21 atributos). Para a comparação dos nove datasets, fixamos 100 iterações em todos os modos, garantindo comparação justa de tempo. No experimento de larga escala (CDC) habilitamos *early stopping* (parada após 20 iterações sem melhora). As métricas são F_1 , taxa de redução, tempo de execução, vazão (GFLOPS) e *speedup*. Para o CDC, o *speedup* é normalizado por iteração, usando como referência (T_1) uma execução de 1 thread da mesma campanha. O código e os dados estão disponíveis publicamente [Soares et al. 2026].

5. Resultados

5.1. Escala pequena/média: a CPU multicore supera a GPU

A Figura 1 e a Tabela 1 mostram o desempenho nas nove bases. A qualidade é preservada por todas as implementações (variações de F_1 entre seq, OpenMP e CUDA ficam dentro de $\sim 2\%$, ruído estatístico do algoritmo estocástico), com redução de 6% a 22% das instâncias. Quanto ao tempo, o *speedup* da CUDA cresce com N (de $0,7\times$ na menor base a $8,2\times$ na maior), mas em **todas** as nove bases o OpenMP em 24 núcleos é mais rápido que a GPU: o custo fixo de lançamento de kernels e de transferência host-device não se amortiza enquanto o problema é pequeno. Em bases de poucos milhares de instâncias, portanto, a CPU multicore é a melhor escolha.

5.2. Escalonadores OpenMP: *dynamic* vence *static*

A avaliação 1-NN é uma carga *irregular* (instâncias com vizinhos próximos terminam mais cedo), o que favorece escalonadores que rebalanceiam a carga. A Figura 2 compara, com 16 threads, os escalonadores `static` (partição em blocos) e `dynamic` (distribuição

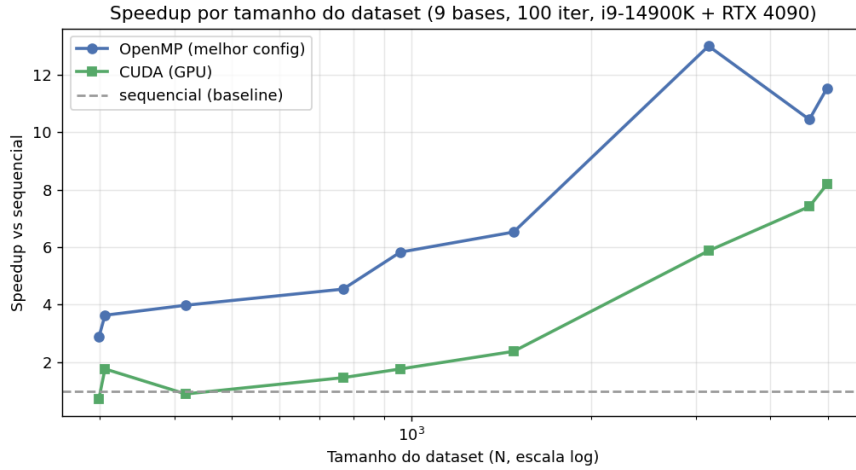


Figura 1. O ganho da GPU cresce com o tamanho da base, mas até ~5k instâncias o OpenMP em 24 núcleos supera a CUDA: o custo fixo da GPU só se amortiza em escala maior.

Tabela 1. Qualidade e *speedup* nas nove bases (100 iterações, sequencial como referência). F_1 do sequencial; melhor configuração OpenMP entre threads e escalonadores.

Dataset	N	F_1	Red. (%)	CUDA	OpenMP (melhor)
heart_failure	299	0,887	19,4	0,7×	2,9×
haberman	306	0,881	19,9	1,8×	3,6×
cirrhosis	418	0,867	21,5	0,9×	4,0×
diabetes	768	0,906	19,5	1,5×	4,5×
tic-tac-toe	958	0,979	6,7	1,8×	5,8×
yeast	1484	0,979	6,3	2,4×	6,5×
vaccine	3152	0,958	12,7	5,9×	13,0×
Employee	4653	0,809	22,2	7,4×	10,4×
brain-stroke	4981	0,727	22,4	8,2×	11,5×

sob demanda) e seus tamanhos de *chunk*. Nas bases maiores, o *dynamic* é em média ~25–30% mais rápido que o *static*, enquanto o tamanho de *chunk* (1, 2 ou 3) tem efeito secundário. O balanceamento dinâmico compensa porque cada iteração do laço quente é pesada o suficiente para diluir o custo da fila compartilhada.

5.3. Larga escala (253.680 instâncias): a GPU domina

Na base CDC, o quadro se inverte. A Tabela 2 resume o desempenho. O OpenMP escala bem com o número de threads, chegando a ~14× (*static*) e ~16× (*dynamic*) sobre 1 thread ao usar as 32 threads do i9-14900K, confirmando que o *dynamic* mantém vantagem em escala. A CUDA, por sua vez, atinge **38–69×** sobre o sequencial, executando a campanha completa em cerca de 2 minutos. A melhor configuração CUDA é ~4,3× mais rápida que a melhor configuração OpenMP (32 threads).

A varredura de tamanho de bloco da CUDA (Figura 3) revela uma curva em U: os extremos (bloco 32 e 1024) são mais rápidos e o meio (64–128) mais lento, com diferença de ~28% entre o melhor e o pior. A explicação está no padrão de acesso do

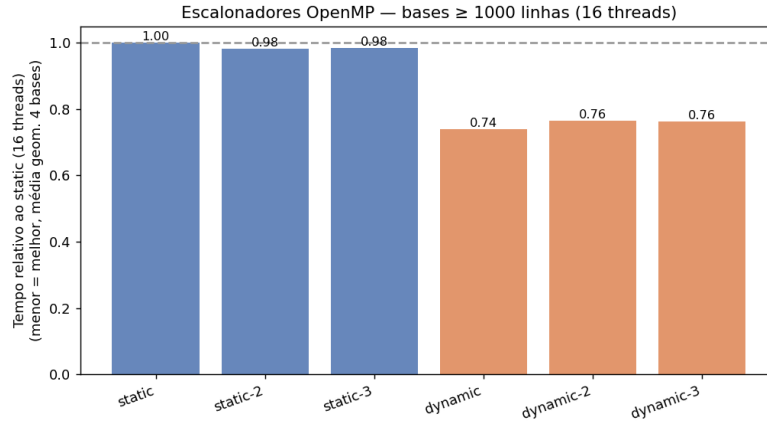


Figura 2. Tempo relativo ao *static* (16 threads, média geométrica das bases ≥ 1000 instâncias): o *dynamic* reduz o tempo em $\sim 25\text{--}30\%$; o tamanho de *chunk* é secundário.

Tabela 2. Desempenho na base CDC (253.680 instâncias; *early stopping* em 22 iterações; *speedup* normalizado por iteração vs. 1 thread). Qualidade idêntica entre os modos.

Modo	Configuração	Tempo	Speedup
OpenMP	2 threads (static)	109,2 min	1,3×
OpenMP	32 threads (static)	9,7 min	14,0×
OpenMP	32 threads (dynamic)	8,6 min	15,9×
CUDA	block size 64	151 s	54,0×
CUDA	block size 1024	118 s	69,1×

knn_1nn_kernel: todas as threads leem a mesma linha de referência simultaneamente, em *broadcast*. Blocos grandes maximizam o reuso desse *broadcast*; blocos pequenos dão folga ao escalonador para manter os SMs cheios; o meio perde nos dois efeitos. O padrão é reprodutível (variação $< 0,2\%$ entre execuções repetidas), confirmando que não é ruído.

Em todos os modos a solução converge para a mesma qualidade: $\sim 23\%$ de redução de instâncias mantendo $F_1 \approx 0,77$ — a paralelização altera o tempo, não o resultado. O *early stopping* reduziu cada execução para 22 iterações, economizando $\sim 78\%$ do tempo sem perda de qualidade.

6. Conclusão

Este trabalho paralelizou o ACO de seleção de instâncias em OpenMP e CUDA, preenchendo a lacuna de escalabilidade deixada pelo baseline sequencial de Magalhães et al. [Magalhães et al. 2024] e viabilizando o algoritmo em uma base de 253.680 instâncias. O resultado central é que **o ganho da GPU é dependente de escala**: em bases de até $\sim 5k$ instâncias, o custo fixo da GPU não se amortiza e o OpenMP em 24 núcleos é mais rápido; em larga escala, a CUDA domina, atingindo $\sim 69\times$ sobre o sequencial e sendo $\sim 4,3\times$ mais rápida que a melhor configuração de CPU. Em todos os casos, a qualidade da seleção é preservada ($\sim 23\%$ de redução, $F_1 \approx 0,77$).

As principais limitações são o uso de uma única base de larga escala e a ausência

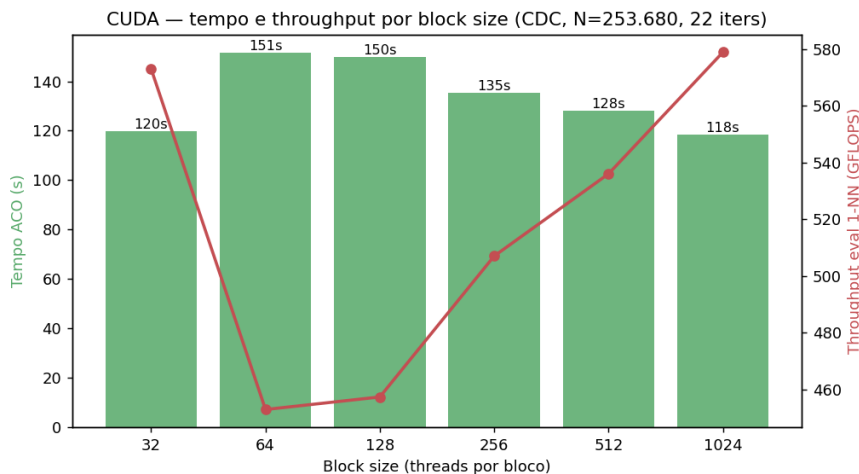


Figura 3. CUDA na base CDC: a curva de tempo e vazão por tamanho de bloco tem forma de U — os extremos vencem por equilibrarem reúso de *broadcast* e ocupância dos SMs.

de *profiling* de hardware (ocupância, vazão de memória) para confirmar quantitativamente a explicação da curva em U. Como trabalhos futuros, pretende-se investigar a *co-execução híbrida* CPU+GPU — particionando o trabalho entre os dispositivos — e estender a avaliação a mais bases de larga escala, aproximando ainda mais o algoritmo de aplicações reais de mineração de dados.

Referências

- Centers for Disease Control and Prevention (2015). CDC diabetes health indicators dataset (BRFSS 2015). UCI Machine Learning Repository. 253.680 instâncias, 21 atributos.
- Cover, T. M. and Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27.
- Dagum, L. and Menon, R. (1998). OpenMP: An industry standard API for shared-memory programming. *IEEE Computational Science and Engineering*, 5(1):46–55.
- Dorigo, M., Birattari, M., and Stützle, T. (2006). Ant colony optimization. *IEEE Computational Intelligence Magazine*, 1(4):28–39.
- Magalhães, J. M. d. O., Gonçalves, A. C. M., Nobre, C. N., and Freitas, H. C. d. (2024). Avaliação de desempenho e escalabilidade do algoritmo de otimização de colônia de formigas em C++ e Python. Código disponível em: <https://github.com/jmarcosjm/aco-cpp>.
- Nickolls, J., Buck, I., Garland, M., and Skadron, K. (2008). Scalable parallel programming with CUDA. *ACM Queue*, 6(2):40–53.
- Olvera-López, J. A., Carrasco-Ochoa, J. A., Martínez-Trinidad, J. F., and Kittler, J. (2010). A review of instance selection methods. *Artificial Intelligence Review*, 34(2):133–143.

Soares, R. A. L., Menezes, W. V. A., Jácome, V. G. R., and Borges, D. R. (2026). aco-selection-parallel: Aco paralelo (openmp e cuda) para seleção de instâncias. <https://github.com/phael-exe/aco-selection-parallel>.